Architecture finale recommandée :

- **Wallet-first**
- **QR-first**
- **NFC plus tard en option**
- **SaaS multi-tenant**
- **backend central unique**
- **interface restaurateur ultra-rapide en PWA**
- **moteur fidélité entièrement côté serveur**
- **mode rush sans scan obligatoire**

Tu dois figer le produit comme ça :

1 carte universelle client
+ programmes par restaurant en backend
+ validation par scan OU recherche rapide OU bouton manuel
+ rewards automatiques
+ analytics commerçant

---

1. Flow réel en restaurant

Cas 1 — usage normal

1. Le client arrive
2. Il montre sa carte Wallet
3. Le staff :
    - scanne le QR
      ou
    - retrouve le client dans “derniers clients”
4. Le backend crédite le bon programme
5. L’écran affiche immédiatement :
    - point ajouté
    - total actuel
    - récompense atteinte si applicable

Temps cible :

- **2 à 4 secondes max**

Cas 2 — rush

1. Le client dit son prénom / téléphone / nom de compte
2. Le staff le retrouve en recherche instantanée
3. Il clique `+1`
4. Le backend traite

Temps cible :

- **moins de 2 secondes**

Cas 3 — récompense

1. Le client a atteint un seuil
2. L'interface affiche `récompense disponible`
3. Le staff clique `utiliser la récompense`
4. Le backend verrouille et consomme la reward

Temps cible :

- **1 à 2 secondes**

---

2. Architecture produit finale

2.1 Côté client

Le client possède :

- une **carte universelle Wallet**
- un **QR fixe signé**
- éventuellement :
  - nom
  - identifiant court
  - message marketing
  - résumé simplifié

Le Wallet sert à :

- identifier le client
- faciliter l'ajout
- pousser des notifications
- éviter une app dédiée

Il ne sert pas à porter toute la logique métier.

2.2 Côté restaurant

Le restaurant dispose d'une **PWA / web app ultra légère** avec 5 fonctions maximum :

- Scanner
- Rechercher client
- Derniers clients
- Ajouter un passage / des points
- Consommer une récompense

L'interface doit être pensée pour :

- téléphone
- tablette
- écran caisse

Pas de dashboard complexe en écran principal.
Le dashboard analytics est séparé.

2.3 Côté plateforme

La plateforme centralise :

- onboarding clients
- onboarding restaurants
- programmes fidélité
- événements de fidélité
- rewards
- analytics
- abonnements

---

3. Architecture finale des couches

Couche 1 — Wallet Layer

Responsable de :

- génération Apple Wallet / Google Wallet
- mise à jour des passes

- notifications Wallet
- templates visuels

Cette couche ne décide pas des points.

Couche 2 — Merchant Ops Layer

Responsable de :

- interface de scan
- recherche client
- mode rush
- actions rapides staff

C’est la couche critique business.
C’est elle qui fait ou casse l’adoption.

Couche 3 — Core Loyalty Engine

Responsable de :

- règles de points
- tampons
- récompenses
- bonus
- validation métier

C’est le cœur du produit.

Couche 4 — Control Layer

Responsable de :

- auth
- multi-tenant
- logs
- sécurité
- analytics
- billing
- administration

---

4. Architecture technique finale

Frontend

Merchant app

- **Next.js**
- **Tailwind**
- **PWA**
- compatible mobile/tablette/desktop

Marketing / landing

- Next.js aussi
- même repo possible au départ

Backend

- **NestJS**
- architecture monolithique modulaire
- API REST
- workers séparés si besoin plus tard

Pourquoi :

- rapide à développer
- propre pour multi-domaines
- bon contrôle métier
- scalable sans microservices trop tôt

Base de données

- **PostgreSQL**

Pourquoi :

- relations fortes
- transactions
- cohérence métier
- très bon pour SaaS multi-tenant

Cache / état transitoire

- **Redis**

Utilisation :

- cooldown
- rate limit
- locks
- cache programme
- sessions temporaires

Infra

- **Docker Compose** au départ
- **Nginx**
- **Cloudflare**
- **VPS solide**
- backups PostgreSQL automatiques
- monitoring basique

---

5. Architecture métier finale

Principe central

Tu ne construis pas une carte par restaurant.
Tu construis :

- **1 client global**
- **N relations marchandes**
- **N comptes fidélité**
- **1 historique d'événements par relation**

Entités principales

customers

Identité globale minimale du client

wallet_passes

Pass Wallet lié au client

merchants

Restaurant / commerce

merchant_locations

Établissements physiques

staff_users

Employés / managers

loyalty_programs

Programme du restaurant

loyalty_accounts

Compte fidélité du client chez ce restaurant

loyalty_events

Chaque crédit / débit / reward / action

rewards

Récompenses acquises / consommées

subscriptions

Abonnement du commerçant

---

6. Schéma logique final

customers
- id
- public_id
- first_name
- phone
- email
- created_at

wallet_passes
- id
- customer_id
- qr_payload
- qr_signature
- apple_pass_id
- google_object_id
- status
- updated_at

merchants
- id
- name
- slug
- status
- created_at

merchant_locations
- id
- merchant_id
- name
- address
- city
- created_at

staff_users
- id
- merchant_id
- location_id
- email
- password_hash
- role
- status
- created_at

loyalty_programs
- id
- merchant_id
- name
- type
- rules_json
- reward_policy_json
- status
- created_at

loyalty_accounts
- id

- merchant_id
- customer_id
- loyalty_program_id
- points_balance
- stamps_balance
- total_earned
- total_redeemed
- last_activity_at
- created_at

loyalty_events
- id
- merchant_id
- customer_id
- loyalty_program_id
- location_id
- staff_user_id
- source
- event_type
- points_delta
- stamps_delta
- metadata_json
- created_at

rewards
- id
- merchant_id
- customer_id
- loyalty_program_id
- status
- reward_type
- reward_value
- earned_at
- redeemed_at
- expires_at

subscriptions
- id
- merchant_id
- plan_code
- status
- trial_ends_at
- current_period_start
- current_period_end

---

7. Sources métier finales à supporter

Chaque crédit de fidélité doit venir d'une source explicite :

- `qr_scan`
- `manual_search_credit`
- `quick_add_recent_customer`
- `reward_redeem`
- `admin_adjustment`
- plus tard :
  - `nfc_tap`
  - `pos_sync`

C'est important pour :

- audit
- anti-abus
- analytics
- évolution du produit

---

8. API finale minimale

Auth

POST /auth/login
POST /auth/refresh
POST /auth/logout

Customer onboarding

POST /customers
GET /customers/:id
GET /customers/search?q=

Wallet

POST /wallet/passes
PATCH /wallet/passes/:customerId
GET /wallet/passes/:customerId

Merchant

GET /merchant/me
GET /merchant/programs
POST /merchant/programs
PATCH /merchant/programs/:id

Loyalty actions

POST /scan
POST /loyalty/credit
POST /loyalty/redeem
GET /loyalty/accounts/:customerId
GET /loyalty/accounts/:customerId/history

Analytics

GET /analytics/overview
GET /analytics/customers
GET /analytics/rewards
GET /analytics/retention

Billing

GET /subscription
POST /subscription/checkout
PATCH /subscription/plan

---

9. Règles métier finales

Ce qu’il faut autoriser

- usage familial
- QR fixe
- recherche client manuelle
- validation par staff

Ce qu’il faut bloquer

- crédits répétés en boucle

- rewards consommées deux fois
- accès cross-tenant
- actions sans trace

Garde-fous minimum

- cooldown par client/resto
- limite journalière paramétrable
- lock transactionnel sur redeem
- journal complet
- risk score léger

---

10. UX finale restaurateur

Écran 1 — caisse / rush

Doit afficher seulement :

- bouton scanner
- barre recherche
- derniers clients
- bouton +1
- reward dispo

Écran 2 — fiche client

- identité simple
- progression fidélité
- historique récent
- récompense disponible
- action rapide

Écran 3 — admin programme

- définir règles :
  - points
  - tampons
  - seuils
  - récompenses

Écran 4 — analytics

- clients actifs
- fréquence
- rewards émises
- rewards consommées
- rétention

---

11. Architecture sécurité finale

Auth

- JWT access/refresh
- rôles staff / manager / owner

Multi-tenant

- `merchant_id` obligatoire partout
- aucune requête métier sans filtre tenant

QR

- QR fixe signé
- validation serveur uniquement

Logs

- tout événement métier loggé
- tout redeem loggé
- toute modif programme loggée

Secrets

- hors code
- rotation possible
- chiffrement des données sensibles si nécessaire

---

12. Architecture d’évolution finale

Phase 1 — MVP

- carte universelle
- QR fixe
- scan
- recherche client
- bouton manuel
- rewards simples
- analytics basiques

Phase 2 — croissance

- push Wallet
- segmentation
- analytics avancées
- onboarding amélioré
- support multi-sites complet

Phase 3 — premium

- NFC
- POS
- automation marketing
- recommandations promos
- intégrations externes

---

13. Décision finale figée

Architecture finale recommandée

- **Wallet universel client**
- **PWA restaurateur**
- **NestJS monolith modulaire**
- **PostgreSQL**
- **Redis**
- **QR fixe signé**
- **mode rush sans dépendance au scan**
- **NFC reporté**
- **SaaS multi-tenant strict**

Pourquoi c’est la bonne architecture

Parce qu’elle optimise simultanément :

- **time-to-market**
- **adoption terrain**
- **simplicité produit**
- **scalabilité**
- **marges**
- **évolutivité premium**

Conclusion

Ton produit final ne doit pas être pensé comme :

- une carte
- un QR
- une techno Wallet

Il doit être pensé comme :

> **une infrastructure de fidélité ultra-rapide pour commerces physiques, avec un canal client simple et une exécution caisse sans friction**

La bonne architecture finale est donc :
QR-first, Wallet-first, backend-first, rush-first.

Document fondateur — Solution de fidélité digitale universelle pour restaurants

1. Nom de travail

Carte de fidélité universelle Wallet-first pour restaurants

Nom produit temporaire :

- **TapLoyal**
- **WalletLoyal**
- **Stamply**
- **Fidelio Wallet**
- **Union Pass**

Nom interne recommandé pour l'instant :
Wallet Loyalty Network

---

2. Vision

Créer l'infrastructure de fidélité la plus simple pour les commerces physiques :

- sans application mobile dédiée
- sans friction client
- sans complexité technique pour le commerçant
- avec une adoption immédiate via Apple Wallet et Google Wallet

Le produit doit devenir :

> **le standard de fidélité digitale local, simple, rapide et universel**

---

3. Problème

Les systèmes actuels ont 3 défauts majeurs :

A. Carte papier

- se perd
- aucune donnée exploitable

- aucune automatisation
- image vieillissante

B. App dédiée

- personne ne veut télécharger une app pour un resto local
- fort abandon
- UX trop lourde
- coût de dev et maintenance élevé

C. Solutions trop complexes

- intégration difficile
- trop chères pour petits commerces
- pas pensées pour le rush en caisse

---

4. Solution

Une **carte de fidélité digitale universelle**, stockée dans Apple Wallet / Google Wallet, permettant à un client de cumuler des avantages dans plusieurs restaurants, avec une gestion individualisée par établissement.

Principe

- 1 client = 1 carte universelle
- 1 restaurant = 1 programme fidélité configurable
- validation = scan QR ou recherche rapide ou bouton manuel
- récompenses = gérées automatiquement côté serveur

---

5. Proposition de valeur

Pour le client

- aucune app à installer
- une seule carte
- utilisable partout dans le réseau
- accès simple et immédiat
- expérience fluide

Pour le restaurateur

- fidélité moderne sans friction
- outil simple pour le staff
- données exploitables
- augmentation de la fréquence client
- mise en place rapide

Pour la plateforme

- effet réseau
- récurrence SaaS
- scalabilité multi-commerces
- potentiel de standard local/régional/national

---

6. Cible initiale

Cible primaire

- restaurants indépendants
- pizzerias
- snacks
- kebabs
- boulangeries
- coffee shops
- petite restauration de quartier

Cible secondaire

- franchises locales
- dark kitchens
- chaînes régionales

Cible future

- retail food
- salons de coiffure
- commerces de proximité
- salles de sport
- services locaux

---

7. Positionnement

Positionnement recommandé :

> **La carte de fidélité moderne, sans application, directement dans le téléphone du client.**

Positionnement commercial court :

> **Plus simple qu’une app. Plus moderne qu’une carte papier.**

---

8. Décision produit clé

Choix final

- **QR-first**
- **Wallet-first**
- **NFC plus tard**
- **mode rush obligatoire**
- **backend centralisé**
- **SaaS multi-tenant**

Ce qu’on ne fait pas au départ

- NFC only
- app mobile dédiée
- POS natif dès le MVP
- IA marketing complexe
- gamification lourde

---

9. Fonctionnement produit

Onboarding client

1. Le client scanne un QR public du restaurant
2. Il ajoute sa carte au Wallet

3. Son compte est créé
4. Le programme du restaurant est lié à son profil

Passage en restaurant

Le staff peut :

- scanner le QR client
- rechercher le client
- créditer via bouton rapide

Récompense

Le système :

- calcule le progrès
- détecte le seuil atteint
- crée la récompense
- permet son utilisation par le staff

---

10. Cas d’usage principal

Exemple pizzeria

- 1 pizza achetée = 1 tampon
- à 10 tampons = 1 pizza offerte

Exemple coffee shop

- 1 café = 1 point
- 10 points = 1 boisson offerte

Exemple snack

- 1 passage = 1 point
- bonus sur créneaux creux

---

11. Architecture produit finale

A. Wallet Layer

Responsable de :

- génération des cartes Apple/Google Wallet
- mises à jour
- notifications push futures

B. Merchant Ops Layer

Responsable de :

- scan QR
- recherche rapide client
- mode rush
- validation et utilisation récompenses

C. Core Loyalty Engine

Responsable de :

- règles fidélité
- points/tampons
- bonus
- récompenses
- historique

D. Control Layer

Responsable de :

- auth
- billing
- sécurité
- analytics
- logs
- administration

---

12. Stack technique finale

Frontend

- **Next.js**
- **Tailwind CSS**
- **PWA** pour interface restaurateur

Backend

- **NestJS**
- API REST
- monolithe modulaire

Base de données

- **PostgreSQL**

Cache / anti-abus / locks

- **Redis**

Infra

- **Docker Compose**
- **Nginx**
- **Cloudflare**
- **VPS**
- backups automatiques

---

13. Schéma métier final

Entités clés

- `customers`
- `wallet_passes`
- `merchants`
- `merchant_locations`
- `staff_users`
- `loyalty_programs`
- `loyalty_accounts`
- `loyalty_events`
- `rewards`
- `subscriptions`

Principe logique

- identité client globale minimale
- relation commerciale séparée
- compte fidélité par restaurant
- événements historisés
- récompenses transactionnelles

---

14. Sources d’actions supportées

Chaque action doit avoir une source claire :

- `qr_scan`
- `manual_search_credit`
- `quick_add_recent_customer`
- `reward_redeem`
- `admin_adjustment`

Plus tard :

- `nfc_tap`
- `pos_sync`

---

15. UX finale restaurateur

Écran principal

Doit contenir uniquement :

- bouton scanner
- champ recherche client
- derniers clients
- bouton `+1`
- récompense disponible

Écran fiche client

- prénom / identifiant
- progression

- historique récent
- bouton `+1`
- bouton `utiliser récompense`

Écran admin

- configuration programme
- seuils
- règles
- récompenses

Écran analytics

- clients actifs
- fréquence
- rewards émises
- rewards consommées
- rétention

---

16. Sécurité et règles métier

Autorisé

- QR fixe
- usage familial
- validation manuelle
- recherche client
- rewards automatiques

Interdit / bloqué

- spam de crédits
- double consommation récompense
- accès cross-tenant
- actions sans logs

Garde-fous

- signature QR
- validation serveur uniquement
- cooldown

- limite journalière
- lock transactionnel sur reward redeem
- audit logs

---

17. Business model

Offre de base

- **29€/mois** : 1 établissement
- **79€/mois** : multi-sites
- **99€ setup** : onboarding / paramétrage / assets de lancement

Upsells

- pack signalétique physique
- analytics avancées
- campagnes push
- programme premium réseau
- white-label
- POS / NFC plus tard

---

18. Pack de lancement recommandé

Inclure dès le départ :

- stickers QR
- chevalet comptoir
- mini guide “comment proposer la carte”
- visuels imprimables
- onboarding restaurant

Ce pack augmente fortement l’adoption.

---

19. KPI à suivre

Produit

- taux d’ajout Wallet
- scans / crédits par client
- récompenses débloquées
- récompenses consommées

Business

- nombre de restos signés
- MRR
- churn
- CAC
- temps moyen onboarding

Rétention

- clients actifs J30/J60/J90
- fréquence visite moyenne
- taux de retour

---

20. Différenciation stratégique

Ce qui te différencie

- pas d’app
- Wallet natif
- 1 carte universelle
- UX rush pensée terrain
- déploiement rapide
- prix accessible

Ce qui devient ton moat

- réseau multi-restaurants
- onboarding déjà fait côté client
- historique fidélité
- simplicité supérieure aux concurrents

---

21. Roadmap produit

Phase 1 — MVP

- carte Wallet universelle
- QR fixe signé
- création client
- programme par restaurant
- scan
- recherche client
- bouton manuel
- rewards simples

Phase 2 — Croissance

- analytics avancées
- segmentation
- amélioration onboarding
- push Wallet
- multi-sites mature

Phase 3 — Premium

- NFC
- POS
- campagnes avancées
- moteur promo intelligent
- réseau inter-commerces

---

22. Go-to-market

Canal principal

- démarchage terrain
- démo immédiate en face-à-face

Canal secondaire

- Instagram / DM
- cold email local
- partenariats fournisseurs locaux
- ambassadeurs commerçants

Offre commerciale recommandée

- essai gratuit 14 jours
- installation offerte si signature rapide
- 1 mois offert par parrainage commerçant

---

23. Risques principaux

Produit

- UX trop lente en rush
- trop de clics
- scan pas assez rapide

Business

- acquisition terrain difficile
- faible discipline staff
- support humain sous-estimé

Technique

- Wallet plus long que prévu
- qualité d'intégration Apple/Google
- dette produit si on ajoute trop vite NFC/POS

---

24. Règle stratégique absolue

Ne jamais oublier :

> **le produit n'est pas le QR**
> **le produit n'est pas le Wallet**
> **le produit n'est pas la techno**

Le produit est :

> **la capacité à créditer un client en moins de 2 secondes, sans friction, dans un commerce physique**

---

25. Décision finale

Architecture finale figée

- **Wallet-first**
- **QR-first**
- **PWA restaurateur**
- **NestJS + PostgreSQL + Redis**
- **multi-tenant strict**
- **mode rush natif**
- **NFC reporté**
- **POS reporté**
- **SaaS scalable**

---

26. Priorités immédiates

Priorité 1

Définir le MVP exact

Priorité 2

Dessiner les écrans restaurateur

Priorité 3

Construire le schéma DB

Priorité 4

Développer le flow onboarding + crédit

Priorité 5

Tester sur 3 à 5 restaurants réels

---

27. Résumé ultra-court à réutiliser

Nous construisons une solution SaaS de fidélité digitale universelle pour restaurants, sans application, basée sur Apple Wallet et Google Wallet. Le client possède une carte universelle unique, et chaque restaurant gère son propre programme de fidélité côté serveur. L’architecture est QR-first, pensée pour le rush en caisse, avec une interface restaurateur ultra-simple, un backend centralisé NestJS/PostgreSQL/Redis, et une roadmap évolutive vers NFC et POS.

PRD — Product Requirements Document (version MVP → scalable)

1. Objectif produit

Construire un MVP permettant :

- onboarder un restaurant en < 10 min
- onboarder un client en < 30 sec
- créditer un client en < 2 sec
- déclencher une récompense automatiquement

👉 KPI principal :
temps d'utilisation réel en caisse

---

2. Users

2.1 Restaurateur

- propriétaire
- manager
- staff (serveur, caissier)

2.2 Client final

- utilisateur Wallet
- usage occasionnel
- aucune formation

---

3. Use cases principaux

UC1 — Ajouter la carte

- scan QR public
- ajout Wallet
- création compte automatique

UC2 — Créditer un client

- scan QR OU recherche OU bouton rapide
- validation immédiate

UC3 — Récompense

- atteinte seuil
- affichage
- utilisation par staff

UC4 — Pilotage

- consulter stats
- config programme fidélité

---

4. User stories

Client

- “Je veux ajouter la carte en 1 scan”
- “Je veux voir mes points”
- “Je veux savoir quand j’ai une récompense”

Staff

- “Je veux créditer un client sans ralentir le service”
- “Je veux éviter les erreurs”
- “Je veux voir si une récompense est dispo”

Owner

- “Je veux voir si ça me rapporte”
- “Je veux configurer mes règles”
- “Je veux garder mes clients”

---

5. Fonctionnalités MVP

5.1 Onboarding client

- QR public restaurant
- création customer
- génération Wallet pass
- liaison restaurant ↔ client

5.2 Wallet

- QR fixe signé
- affichage simple
- mise à jour possible

5.3 Crédit fidélité

- scan QR
- recherche client
- bouton manuel
- feedback immédiat

5.4 Programme fidélité

- type : points ou tampons
- seuil reward
- règles simples

5.5 Rewards

- création automatique
- statut : available / redeemed
- consommation staff

5.6 Dashboard simple

- nombre clients
- nombre passages
- rewards délivrées

---

6. Non-fonctionnel

- latence API < 300ms

- disponibilité > 99%
- multi-tenant strict
- mobile-first
- UX utilisable sans formation

---

7. Flow UX (écran par écran)

7.1 Écran caisse (principal)

- bouton SCAN
- champ RECHERCHE
- liste DERNIERS CLIENTS
- bouton +1
- badge reward dispo

7.2 Scan

- ouverture caméra
- lecture QR
- réponse instant

7.3 Recherche

- recherche temps réel
- affichage liste
- sélection client

7.4 Fiche client

- prénom
- progression
- bouton +1
- bouton “utiliser récompense”

---

8. Edge cases critiques

- client sans réseau
- staff sans connexion stable

- double scan
- reward utilisée 2 fois
- client inexistant
- QR invalide

---

9. Schéma DB production-ready (simplifié SQL)

```
CREATE TABLE customers (
  id UUID PRIMARY KEY,
  public_id TEXT UNIQUE,
  first_name TEXT,
  phone TEXT,
  email TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE merchants (
  id UUID PRIMARY KEY,
  name TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE loyalty_programs (
  id UUID PRIMARY KEY,
  merchant_id UUID,
  type TEXT,
```

```
  rules JSONB,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE loyalty_accounts (
  id UUID PRIMARY KEY,
  merchant_id UUID,
  customer_id UUID,
  points INT DEFAULT 0,
  stamps INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE loyalty_events (
  id UUID PRIMARY KEY,
  merchant_id UUID,
  customer_id UUID,
  points_delta INT,
  source TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE rewards (
  id UUID PRIMARY KEY,
  merchant_id UUID,
  customer_id UUID,
```

```
  status TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);
```

---

10. API essentielle (exécutable)

Crédit client

POST /loyalty/credit

```
{
  "customer_id": "...",
  "merchant_id": "...",
  "source": "qr_scan"
}
```

Recherche client

GET /customers/search?q=amine

Redeem reward

POST /rewards/{id}/redeem

---

11. Pitch deck (structure prête)

Slide 1 — Problème

- fidélité cassée
- apps inutilisées
- cartes papier obsolètes

Slide 2 — Solution

- carte Wallet universelle
- sans app
- ultra simple

Slide 3 — Produit

- QR + scan + bouton
- rewards auto
- dashboard simple

Slide 4 — Marché

- millions de commerces physiques
- rétention = enjeu clé

Slide 5 — Business model

- SaaS abonnement
- upsell marketing

Slide 6 — Différenciation

- no app
- universel
- UX rush

Slide 7 — Go-to-market

- terrain
- démo live
- onboarding rapide

Slide 8 — Roadmap

- MVP → scale → NFC/POS

Slide 9 — Vision

- réseau fidélité global

---

12. Plan d’exécution immédiat

Jour 1–2

- finaliser modèle DB
- définir règles fidélité

Jour 3–5

- backend NestJS (core endpoints)
- création customer + credit

Jour 6–7

- interface PWA minimale
- test réel

Semaine 2

- Wallet pass
- onboarding client

Semaine 3

- tests terrain
- itérations UX

---

13. Conclusion

Tu as maintenant :

- architecture complète
- PRD
- DB
- API
- UX
- business model
- roadmap
- pitch

👉 Tu peux build immédiatement.